

Manual de Sistema

Proyecto: UPTP-336-Proyecto-II/Lufra-2020

Descripción: Grupo 1

Repositorio: <https://github.com/UPTP-336-Proyecto-II/Lufra-2020>

1. Información General del Sistema

- **Lenguajes principales:**
 - JavaScript (33.3%)
 - TypeScript (27.3%)
 - Vue (16%)
 - PHP (12.2%)
 - Blade (7.4%)
 - CSS (3.8%)
- **Frameworks y herramientas:**
 - Laravel (supuesto por el uso de PHP y Blade)
 - Vue.js
 - Node.js (npm o yarn para gestión de paquetes)
 - Composer (gestor de dependencias PHP)

2. Descripción del Sistema

“El sistema Lufra-2020” es una aplicación web desarrollada como proyecto académico. Presenta una arquitectura de frontend moderno (Vue.js/JavaScript/TypeScript) que interactúa con un backend robusto (PHP/Laravel). La app está diseñada para ejecutarse en un entorno de servidor web local en desarrollo o en servidores de producción compatibles.

3. Requisitos del Sistema

3.1 Requisitos de Hardware

- **Procesador:** 1 GHz o superior
- **RAM:** al menos 2 GB
- **Espacio en disco:** mínimo 500 MB libres

3.2 Requisitos de Software

- **Servidor web:** Apache o Nginx
- **PHP:** Versión 7.4 o superior
- **Composer:** Última versión
- **Node.js y npm:** Node.js v14 o superior, npm v6 o superior (o Yarn)
- **Base de datos:** MySQL/MariaDB (según configuración)

4. Instalación del Sistema

4.1 Clonar el repositorio

1. Abre una terminal y ejecuta:

Code

- `git clone https://github.com/UPTP-336-Proyecto-II/Lufra-2020.git`

- Ingresa al directorio del proyecto:

Code

2. `cd Lufra-2020`

4.2 Instalar dependencias de backend (PHP)

1. Asegúrate de tener Composer instalado.

2. Ejecuta:

Code

2. `composer install`

4.3 Instalar dependencias de frontend (Node.js)

1. Asegúrate de tener Node.js y npm (o yarn) instalados.

2. Ejecuta:

Code

`npm install`

o

Code

2. `yarn install`

4.4 Configuración de entorno

1. Copia el archivo de ejemplo de variables de entorno:

Code

- `cp .env.example .env`

- Edita el archivo `.env` para configurar los siguientes parámetros:

- **APP_NAME:** Nombre de la aplicación
- **APP_URL:** URL de la app (por ejemplo, <http://localhost>)
- **DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD:** Datos de conexión a la base de datos

- Genera la clave de la aplicación de Laravel:

Code

- `php artisan key:generate`

- Ejecuta las migraciones (si hay):

Code

```
4. php artisan migrate
```

4.5 Compilación del Frontend

Ejecuta:

Code

```
npm run dev
```

o

Code

```
yarn dev
```

- Si quieres compilar para producción, usa:

Code

```
npm run build
```

o

Code

- yarn build

5. Estructura de Archivos, Directorios y Módulos

Nota: Los nombres y descripciones de los módulos pueden necesitar ajuste según la estructura real del repositorio.

- **/app**
Lógica principal del backend (controladores, modelos, políticas, etc.).
- **/resources/views**
Plantillas Blade para la interfaz generada por Laravel.
- **/resources/js**
Componentes front-end desarrollados con Vue.js u otra tecnología JS.
- **/database/migrations**
Archivos para crear y modificar la estructura de la base de datos.
- **/routes/web.php**
Define las rutas del sistema web (enrutamientos principales a los controladores).
- **/public**
Archivos públicos (index.php, imágenes, JS y CSS compilados, favicon, etc.).

- **/config**
Archivos de configuración de Laravel.
- **/src (si existe)**
Código fuente adicional para frontend moderno (Vue/TypeScript).

5.1 Módulos Principales (Ejemplo)

- **Módulo de Gestión de Usuarios**
Permite el registro, inicio de sesión, edición, listado y eliminación de usuarios.
- **Módulo de Administración de Productos o Recursos**
CRUD para productos, servicios o recursos principales de la aplicación.
- **Módulo de Reportes**
Generación de informes sobre el uso del sistema, exportación a PDF o Excel.
- **Módulo de Panel Administrativo**
Interfaz para usuarios administradores, gestión de tablas maestras y parámetros del sistema.
- **Módulo de Integración y Notificaciones**
Gestión de notificaciones por correo o integraciones con otros sistemas (según configuración).

Importante: Corrige o agrega módulos según lo que observes en `/app/Http/Controllers`, `/resources/js/components` y otros directorios relevantes del proyecto.

6. Base de Datos

6.1 Motor y Conexión

- **Motor:** MySQL o MariaDB (por defecto en Laravel, ajustable en `.env`)
- **Configuración de conexión:**
Define los parámetros en `.env`:
 - `DB_CONNECTION=mysql`
 - `DB_HOST=127.0.0.1`
 - `DB_PORT=3306`
 - `DB_DATABASE=lufra_db` (o el nombre asignado)
 - `DB_USERNAME=root` (o tu usuario)
 - `DB_PASSWORD=` (contraseña)

6.2 Principales Tablas (Ejemplo basado en sistemas típicos)

- **users**
Guarda usuarios del sistema (id, name, email, password, role, timestamps).
- **productos**
Información de productos/servicios gestionados ([id], nombre, descripción, precio, etc.).

- **orders (o pedidos/ventas)**
Registro de transacciones (id, user_id, producto_id, cantidad, estado, fecha).
- **notifications**
Notificaciones internas para los usuarios.

6.3 Migraciones

Laravel gestiona la estructura de la base de datos con migraciones en `/database/migrations`:

Para aplicar la estructura en una base de datos nueva:

Shell

```
php artisan migrate
```

6.4 Relaciones

Suelen establecerse relaciones en los modelos:

- Un usuario puede tener muchos pedidos.
 - Un pedido puede incluir uno o varios productos.
-

7. Estructura Interna y Flujo de Datos

7.1 Ciclo típico de operación

1. **El usuario accede a la plataforma**
Se muestra la pantalla de login o el panel principal según el estado de sesión.
2. **Acceso a los módulos**
El frontend (Vue.js) realiza llamadas (AJAX o fetch) a los endpoints definidos en Laravel (rutas `/api` o `/web`).
3. **Procesamiento en backend**
Laravel recibe la petición, procesa la lógica (controlador → modelo → base de datos), y retorna la respuesta (generalmente JSON o vista Blade).
4. **Despliegue de información**
El frontend actualiza la interfaz usando los datos recibidos.

7.2 Componentes de Comunicación

- **Frontend:** Vue.js, TypeScript, JavaScript en `/resources/js` o `/src`.
- **Backend:** Laravel (PHP), controladores en `/app/Http/Controllers`.
- **Base de datos:** ORM Eloquent, consultas y persistencia en modelos `/app/Models` o `/app`.

7.3 Rutas y Controladores

- Las rutas web están definidas en `/routes/web.php`.
 - Las rutas API (si existen) están en `/routes/api.php`.
 - Cada ruta apunta a un controlador que dirige el flujo de la aplicación.
-

8. Actualización

- Para actualizar el proyecto a la última versión del repositorio remoto:

Code

- `git pull origin main`

- Repite instalación de dependencias si es necesario:

Code

- `composer install`
`npm install`
-

9. Mantenimiento y Soporte

- **Actualizaciones:** Mantén actualizado PHP, Node.js, Composer y los paquetes del sistema.
- **Limpieza:** Borra caches y archivos temporales regularmente usando:

Code

- `php artisan cache:clear`
`npm cache clean --force`

- **Soporte:** Reporta incidencias o consulta dudas en la sección “Issues” del repositorio GitHub.
-

10. Contacto

- Para soporte técnico, reportar bugs o solicitar características nuevas, utiliza el apartado Issues en el repositorio: <https://github.com/UPTP-336-Proyecto-II/Lufra-2020/issues>